

Deep Learning Systems Analysis Report

Deep Learning for Player Action Prediction from Atari Gameplay Frames

Student: Robert Mayfield

Course: Udacity AI Masters Capstone, Deep Learning Systems

Report Overview

This report presents a behavioral cloning experiment (Pomerleau, 1989) that trains convolutional neural networks on the Atari-HEAD human gameplay dataset (Zhang et al., 2020) to predict player actions from Montezuma’s Revenge gameplay frames. The model architecture follows the DQN convolutional design (Mnih et al., 2015), adapted for supervised classification over five action classes derived from real human demonstrations. A controlled comparison tests whether four stacked consecutive frames improve action prediction over a single static frame, with exactly one variable (the input representation) changing between the baseline and experimental models. A supplemental experiment examines the effect of boosted NOOP class weighting on idle frame detection performance.

Dataset and Task Description

The dataset is Atari-HEAD (Zhang et al., 2020), a collection of real human gameplay recordings from Atari 2600 games. Atari-HEAD captures synchronized eye-tracking data, gameplay frames, and action inputs from a single human player across 27 play sessions. The full Atari-HEAD study collected data from four research participants across 20 Atari games; this project uses only the Montezuma’s Revenge recordings from one participant. The Montezuma’s Revenge subset used in this project contains 27 sessions (20 standard and 7 highscore) totaling 447,300 frames. Each frame is paired with the action the player pressed at that moment, recorded as an Arcade Learning Environment (ALE) action code. The dataset is publicly available under CC BY 4.0 and was retrieved from Zenodo.

Montezuma’s Revenge was selected over other Atari titles for three reasons. First, it is a side scrolling platformer requiring jumping, ladder climbing, key collection, and hazard avoidance, making its action vocabulary directly relevant to platformer behavior modeling. Second, its class distribution is far more balanced than alternatives such as Breakout, where the NOOP action accounts for over 78% of frames. Third, its 447,300 frame count provides sufficient scale for deep learning training on GPU hardware.

ALE action codes were mapped to five simplified classes representing the meaningful player actions in the game: NOOP (idle or fire with no direction), RIGHT, LEFT, UP (climb up or jump), and DOWN (crouch or climb down). The resulting class distribution is: LEFT 27.8%, NOOP 26.4%, RIGHT 25.7%, DOWN 10.6%, UP 9.5%. The dataset represents a single player’s behavioral signature across many play sessions at varying skill levels.

All 447,300 frames are stored as 84x84 grayscale uint8 arrays in CPU RAM (approximately 2.94 GB), with normalization to float32 deferred to batch load time to avoid an out of memory condition during dataset construction.

Model Architecture and Design Decisions

Architecture

Both the baseline and experimental models use the same ActionCNN architecture, derived from the DQN convolutional design (Mnih et al., 2015). The architecture consists of three convolutional layers followed by two fully connected layers:

- Conv2d(in_channels, 32, kernel 8x8, stride 4) + ReLU
- Conv2d(32, 64, kernel 4x4, stride 2) + ReLU
- Conv2d(64, 64, kernel 3x3, stride 1) + ReLU
- Linear(3136, 512) + ReLU
- Dropout(p=0.5)
- Linear(512, 5)

At 84x84 input, the three convolutional layers reduce spatial dimensions to a 7x7x64 feature map, which flattens to 3,136 features before the classification head. The flat feature size is computed dynamically, making the class portable to other input resolutions. The baseline model (in_channels=1) has 1,680,549 parameters. The experimental model (in_channels=4) has 1,686,693 parameters.

Architectures Considered

Three architecture families were evaluated before selecting the DQN CNN.

Recurrent Neural Network (LSTM). An LSTM processes frames sequentially and maintains hidden state across timesteps, enabling long range temporal dependencies (Goodfellow et al., 2016, chap. 10). This was rejected for two reasons. First, the temporal context needed for this task is short: four consecutive frames are sufficient to encode character motion direction. Frame stacking handles this more efficiently than a recurrent component and adds no training complexity. Second, an LSTM applied frame by frame introduces sequential dependencies that complicate batching and slow training relative to a fully parallel CNN.

Vision Transformer (ViT). A ViT applies self attention across patches of the input frame, enabling global context across the full image. This was rejected because ViTs trained from scratch require ImageNet-scale data to match CNN performance; Dosovitskiy et al. (2021) recommend large scale pretraining for ViT on smaller downstream tasks, and 447,300 frames is insufficient to realize its advantage. In addition, global self attention across all patch pairs is a computationally expensive solution to a local pattern detection problem that CNNs handle efficiently through translation invariant filters. Using the DQN CNN also keeps results directly comparable to the Atari deep learning literature.

Pretrained CNN (ResNet). A ResNet pretrained on ImageNet could provide strong feature extraction without training from scratch. This was rejected because ImageNet pretrained weights encode statistical properties of natural photographs (texture, color, object scale) that do not transfer to 84x84 grayscale Atari game frames, which have distinct visual statistics: flat color regions, hard edges, sprite based geometry, and no photographic texture (Goodfellow et al., 2016, chap. 15). Fine tuning a pretrained ResNet on this domain would require significant adaptation and introduces more variables than training the DQN architecture from scratch.

Selected architecture: DQN 3-conv CNN. The DQN architecture was designed specifically for 84x84 Atari frame input and is the established benchmark in the Atari deep learning literature (Mnih et al., 2015). Its translation invariant convolutional filters (Goodfellow et al., 2016, chap. 9) are well suited to detecting local spatial features such as character position, ladder edges, and platform geometry. The 7x7x64 feature map produced at 84x84 input is a well validated design point, and the approximately 1.68 million parameter count is appropriately sized for 447,300 training frames: expressive enough to learn meaningful action predictors without being overparameterized for the dataset scale.

Input Representation

The baseline model accepts a single 84x84 grayscale frame (`in_channels=1`), providing only a static snapshot of game state. The experimental model accepts four consecutive frames stacked along the channel dimension (`in_channels=4`), providing the model with implicit information about motion direction and speed. This multi frame input approach was introduced by Mnih et al. (2015) as a way to encode short range temporal dynamics without requiring a recurrent component.

Training Configuration

Both models are trained with the following fixed settings:

- Optimizer: Adam, learning rate $1e-4$ (Kingma & Ba, 2015)
- L2 regularization: weight decay $1e-4$
- Loss: class weighted CrossEntropyLoss, weights inversely proportional to class frequency (He & Garcia, 2009)
- Regularization: Dropout $p=0.5$ between fully connected layers (Srivastava et al., 2014)
- Epochs: 30
- Batch size: 64
- Hardware: NVIDIA GeForce RTX 3080 (CUDA 12.4)

Data Splits

Sessions are split at the session level to prevent data leakage between training and evaluation. No frame from any session appears in more than one split. The 19/4/4 session allocation produces approximately 70/15/15 proportions by frame count: 314,929 training frames, 65,620 validation frames, and 66,751 test frames.

Class weights are computed from training labels only using inverse frequency weighting, ensuring the minority UP and DOWN classes receive proportional gradient influence without any information from validation or test labels.

Regularization

An initial 30 epoch training run revealed clear overfitting: training loss fell near 0.12 while validation loss diverged above 0.50, with a train/validation accuracy gap exceeding 7 percentage points in a preliminary run before regularization was applied. Dropout ($p=0.5$) and L2 weight decay ($1e-4$) were added to address this, reducing the gap to approximately 4 percentage points and stabilizing validation loss.

Experimental Comparison

The controlled comparison tests one hypothesis: does four frame temporal context improve action prediction over a single static frame?

What changes: The input channel count increases from 1 to 4. Four consecutive gameplay frames are stacked along the channel dimension. The SessionAwareStackedDataset ensures that four frame windows never span session boundaries, preventing a frame from the end of one session from being paired with frames from the start of the next.

What stays constant: Architecture depth, kernel sizes, strides, hidden layer size, dropout rate, optimizer, weight decay, loss function, class weights, number of epochs, batch size, and the train/validation/test split.

Why this comparison is valid: Changing exactly one variable between two otherwise identical models isolates the effect of that variable. Any difference in test set performance can be attributed to temporal context rather than architectural or training differences.

Experiment 2: Boosted NOOP class weight. After evaluating the baseline and stacked models, analysis of the confusion matrix revealed that NOOP had the highest false negative count in both models, nearly

double that of any other class. A third model was trained using the baseline architecture with the NOOP class weight multiplied by 2.0 above its balanced value, all other settings held constant. This experiment tests whether a targeted weight adjustment can improve NOOP recall without reducing overall performance.

Results and Interpretation

Model Comparison

Macro F1 is the primary metric because the five action classes are not perfectly balanced: UP and DOWN each represent roughly 10% of frames while LEFT, NOOP, and RIGHT each represent roughly 25 to 28%. Macro F1 averages the per class F1 scores without weighting by support, so it treats a failure on UP or DOWN as equally important as a failure on LEFT or RIGHT. Accuracy alone would reward a model that ignores minority classes, making it a misleading signal for this task (He & Garcia, 2009).

Metric	Baseline (1-frame)	NOOP-Boosted (1-frame)	Stacked (4-frame)
Accuracy	86.6%	87.4%	90.6%
Macro F1	86.7%	87.5%	90.5%

Per-Class F1 (Stacked Model)

Class	Precision	Recall	F1
NOOP	0.917	0.877	0.897
RIGHT	0.901	0.911	0.906
LEFT	0.926	0.909	0.917
UP	0.853	0.939	0.894
DOWN	0.889	0.932	0.910

Interpretation

The four frame stacked model outperforms the single frame baseline by 4.0 percentage points on accuracy (90.6% vs 86.6%) and 3.8 percentage points on macro F1 (90.5% vs 86.7%). The near identical accuracy and macro F1 scores for both models confirm that the class distribution is balanced enough that neither metric is misleading on its own.

The improvement is consistent across all five action classes. UP recall improved from 0.93 to 0.94 and DOWN recall from 0.90 to 0.93. This supports the hypothesis that temporal context helps most for motion dependent actions. LEFT showed the largest F1 improvement (+5.3pp), with RIGHT close behind (+5.1pp), confirming that horizontal direction of travel is encoded in motion rather than any single frame. LEFT achieved the highest F1 score in the stacked model at 0.92. UP at 0.89 is the only class below 0.90, reflecting the challenge of predicting less frequent vertical actions from limited temporal context.

The NOOP boosted experiment produced a substantial improvement in NOOP recall: from 0.858 to 0.916 (+5.8pp), with NOOP F1 rising +1.3pp and overall macro F1 improving by +0.8pp. However, the stacked model outperforms the NOOP boosted baseline by 3.1pp on accuracy and 3.0pp on macro F1. Weight adjustment helps by penalizing NOOP misses more during training, but temporal context from consecutive frames is a more effective solution for distinguishing idle frames from directional movement.

Non-Technical Summary

The model learns to watch Atari gameplay and predict what button the player is pressing. When the model sees only a single screenshot, it achieves about 87% accuracy: it knows roughly what the player is doing, but it cannot tell which direction they are moving from a frozen image. When the model sees four screenshots

in sequence, accuracy rises to about 91%. Increasing the importance of idle frame detection through class weight adjustment also helped. Accuracy pushed to about 87%, but temporal context from four frames produced the larger and more consistent gain across all action types. The gap between one frame and four frame input is the gap between reading a photograph and watching a short video clip.

Error Analysis

The stacked model misclassifies 9.4% of test frames (6,292 of 66,739), compared to the baseline’s 13.4% error rate. The confusion patterns are highly structured.

NOOP is the single largest source of remaining errors. NOOP predicted as RIGHT and NOOP predicted as LEFT are the top two confusion pairs, together accounting for roughly 26.9% of all errors. An idle player standing in a neutral pose near a platform edge produces frames that can resemble the early frames of a horizontal movement, and even four consecutive stationary frames can look like the startup phase of directional movement.

LEFT-RIGHT confusion, the dominant error mode in the baseline where LEFT predicted as RIGHT was the single largest confusion pair (1,685 cases), is substantially reduced in the stacked model (550 cases). Four consecutive frames encode direction of travel, resolving most of the ambiguity a static snapshot cannot. Cases that remain are likely mid reversal frames where the player stops one direction and begins another within the four frame window.

UP and DOWN errors do not appear among the top confusion pairs, consistent with their strong recall scores (UP: 0.94, DOWN: 0.93). Ladder traversal involves distinctive postures and positions that are visually separable even at 84x84 resolution.

The remaining errors in the stacked model are concentrated at the NOOP vs. action boundary rather than among directional distinctions. A hierarchical cascade architecture (Stage 1: binary idle vs. active; Stage 2: four class directional) would target this specific failure pattern by giving each stage a simpler and better defined decision boundary.

Limitations and Risks

Single subject data. All 447,300 frames represent one player’s behavioral signature. The model learns this player’s specific tendencies: their reaction timing, preferred routes, and action patterns. Performance on frames from a different player is unknown and likely lower. Generalization across players would require a multi subject dataset.

Static session split. The 19/4/4 session split was generated with a fixed random seed. Different random seeds could produce different splits, and performance may vary across splits. No cross validation was performed due to training time constraints.

Short temporal window. Four consecutive frames encode motion over a brief window at approximately 60fps (roughly 67ms). Longer behavioral sequences such as planning a route across multiple rooms, responding to enemy positions, or recovering from a fall are not captured by the four frame window. An LSTM applied on top of CNN features could address longer range dependencies (Goodfellow et al., 2016, chap. 10; Oh et al., 2015).

Resolution and information loss. Resizing frames from 160x210 to 84x84 discards fine grained spatial detail. Some visual cues such as enemy facial direction or subtle platform edge geometry may be lost at this resolution.

Data collection methodology dependency. Atari-HEAD uses a semi frame by frame collection mode in which the game pauses until the player provides input, eliminating the reaction time offset present in standard continuous gameplay recordings (Zhang et al., 2020). This approach produces clean label alignment but is

not compatible with live uncontrolled gameplay. Any future data collection for a new game title, whether for fine tuning or retraining, would need to replicate this controlled methodology. This adds significant infrastructure overhead and precluding the use of existing uncontrolled gameplay recordings or telemetry streams as training data.

Ethical and Responsible Use

Behavioral surveillance concern. A model trained to predict player actions from gameplay frames could in principle be used to profile individual players at a level of granularity beyond what players consent to when playing a game. Applying this technique at scale to monitor or rank players without disclosure and specific consent would be an inappropriate use of the technology.

Single player representation. The model reflects one player’s behavioral patterns, which may encode individual habits, skill level, or cognitive style. Using predictions from this model to make decisions affecting other players (such as matchmaking, difficulty adjustment, or behavioral flagging) without validation on those players would be a misapplication.

Recommended use constraints. Appropriate uses of this model include research into player behavior modeling, prototyping adaptive game director systems with disclosed behavioral tracking, and academic study of action prediction from gameplay. Any production use should involve informed consent, multi subject validation, and transparent disclosure of what behavioral data is collected and how it is used.

Mitigation steps taken. The dataset (Atari-HEAD) was collected under informed consent from research participants as part of a published study (Zhang et al., 2020). No personally identifiable information is included in the frame data. The model is evaluated on a held out test set to assess generalization. Its limitations are documented explicitly to prevent overconfident deployment.

Future Improvements

Hierarchical cascade classifier. The dominant remaining error pattern (NOOP vs. action) motivates a two stage architecture: Stage 1 trains a binary classifier on idle vs. active frames, and Stage 2 trains a four class directional classifier that only sees active frames. Each stage faces a simpler and better defined decision boundary, potentially eliminating the precision-recall tension that limits flat five class performance on NOOP.

Multi subject generalization. Training on gameplay from multiple players and evaluating cross player transfer would substantially increase the practical applicability of the approach. The Atari-HEAD dataset includes data from multiple subjects across other games; a multi subject extension using the same Montezuma’s Revenge game would be a natural next step.

Longer temporal context. An LSTM or temporal attention module applied on top of the CNN feature extractor could capture behavioral sequences spanning more than four frames, enabling prediction of higher-level intent such as navigating toward a key or retreating from an enemy (Oh et al., 2015).

Larger input resolution. Training at 160x210 (original ALE resolution) or with a ResNet feature extractor pretrained on natural images could improve spatial feature quality, particularly for distinguishing subtle enemy or object positions.

Future Integration

The player action predictor trained in this project could serve as a behavioral sensor module within a larger adaptive game director system. The stacked model produces a five class softmax output that estimates the probability of each action at each frame. A game director could consume this signal to detect behavioral

patterns such as prolonged NOOP sequences indicating player confusion, repeated directional oscillation indicating navigation difficulty, or sustained UP and DOWN activity indicating active ladder traversal. These signals could trigger adaptive responses such as hint display, difficulty adjustment, or content modification.

The model’s practical integration constraints should be acknowledged. It was trained on a single player and would require fine-tuning or retraining on the target player population before deployment. Real time inference at 60fps is feasible on the RTX 3080 hardware used for training, but a lightweight deployment version would be needed for CPU or embedded environments. The five class action vocabulary is specific to Montezuma’s Revenge and would need remapping for a different game title.

References

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderoeder, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2010.11929>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Convolutional networks. In *Deep Learning* (chap. 9). MIT Press. <https://www.deeplearningbook.org/contents/convnets.html>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Sequence modeling: Recurrent and recursive nets. In *Deep Learning* (chap. 10). MIT Press. <https://www.deeplearningbook.org/contents/rnn.html>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Representation learning. In *Deep Learning* (chap. 15). MIT Press. <https://www.deeplearningbook.org/contents/representation.html>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1412.6980>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://storage.googleapis.com/deepmind-media/dqn/DQNNaturePaper.pdf>
- Oh, J., Guo, X., Lee, H., Lewis, R. L., & Singh, S. (2015). Action-conditional video prediction using deep networks in Atari games. *Advances in Neural Information Processing Systems*, 28. <https://arxiv.org/abs/1507.08750>
- Pomerleau, D. A. (1989). ALVINN: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems*, 1, 305–313. <https://proceedings.neurips.cc/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87/Paper.pdf>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958. <https://jmlr.org/papers/v15/srivastava14a.html>
- Zhang, R., Walshe, C., Liu, Z., Guan, L., Muller, K., Whritner, J., Zhang, L., Hayhoe, M., & Ballard, D. (2020). Atari-HEAD: Atari human eye-tracking and demonstration dataset. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04), 6811–6820. <https://doi.org/10.1609/aaai.v34i04.6161>